Opis implementacji: travelling-thief-metaheuristics

Dokument opisuje, co zostało zrobione w projekcie, i krok po kroku wyjaśnia najważniejsze fragmenty kodu.

Spis treści

1. Stan projektu
2. Problem w skrócie
3. Przepływ programu
4. Dane: Item, City, Problem
5. Loader: wczytywanie pliku .ttp
6. Distance: macierz odległości
7. Rng: losowość z seedem
8. Individual: osobnik
9. PackingHeuristic: wybór przedmiotów
10. Evaluator: funkcja oceny
11. Algorytm losowy
12. Algorytm zachłanny
13. Mutacje: swap i inwersja
14. Krzyżowanie: OX i PMX
15. Statystyki
16. Config: parametry algorytmów
17. Selekcja: turniej i ruletka
18. Algorytm ewolucyjny
19. Symulowane wyżarzanie (SA)
20. Logowanie i wykresy
21. main.cpp: dwa tryby pracy
22. run_experiments.py: serie eksperymentów
23. Wyniki
24. Założenia
25. Pytania, które mogą paść na obronie

1. Stan projektu

Część 1

- ✅ Zapoznanie się z problemem
- ✅ Implementacja loadera
- ✅ Implementacja losowego osobnika TTP
- ✅ Implementacja funkcji f(x)
- ✅ Implementacja algorytmu zachłannego

Część 2

- ✅ Implementacja operatorów mutacji i krzyżowania
- ✅ Implementacja selekcji i zarządzania populacją w algorytmie ewolucyjnym
- ✅ Implementacja narzędzi logowania → wykresy
- ✅ Implementacja innej metaheurystyki (SA lub TS)
- ✅ Wstępne strojenie metaheurystyk
- ✅ Badanie wpływu wartości parametrów na skuteczność/efektywność algorytmu genetycznego
- ⬜ Przygotowanie sprawozdania: tabelki, wykresy, wnioski

---

2. Problem w skrócie

Złodziej odwiedza **każde miasto dokładnie raz** i wraca do miasta startowego. W miastach leżą przedmioty z **zyskiem** i **wagą**. Plecak ma pojemność `W`.

Haczyk: im cięższy plecak, tym **wolniej** idzie złodziej:

```
v = vmax - Wc * (vmax - vmin) / W
```

- `Wc` to aktualna waga plecaka,
- pusty plecak: `v = vmax` (np. 1.0),
- pełny plecak: `v = vmin` (np. 0.1, czyli 10 razy wolniej).

Cel: **maksymalizować**

```
G = suma_zysków - całkowity_czas_podróży
```

Rozwiązanie składa się z dwóch części: **trasy** (kolejność miast) i **planu przedmiotów** (co zabrać).

---

3. Przepływ programu

```
plik .ttp
   │
   ▼
loadProblem()            ──► Problem (miasta, przedmioty, parametry)
   │
   ▼
Distance(problem)        ──► macierz odległości liczona raz
   │
   ▼
Evaluator(problem, distance)
   │
   │   dla każdego osobnika (trasy):
   │      packItems()    ──► które przedmioty zabrać
   │      symulacja      ──► G = zysk - czas
   ▼
RandomSearch / Greedy / EA / SA   ──► wiele ocen ──► computeStats() ──►
results/wyniki.csv
   │
   └─ historia przebiegu EA / SA ──► results/history*/  ──► Python: wykresy, tabela
```

Plik	Odpowiedzialność
`Item.h`, `Problem.h`	struktury danych
`Loader`	wczytanie pliku `.ttp`
`Distance`	odległości między miastami
`Rng.h`	generator liczb losowych z seedem
`Individual`	osobnik (trasa + plan przedmiotów)
`PackingHeuristic`	zachłanny wybór przedmiotów dla trasy
`Evaluator`	funkcja oceny `G` + licznik ocen
`solvers/RandomSearch`	metoda losowa
`solvers/Greedy`	metoda zachłanna (najbliższy sąsiad)
`operators/Mutation`	mutacje swap i inwersja
`operators/Crossover`	krzyżowanie OX i PMX
`operators/Selection`	selekcja: turniej i ruletka
`Config`	parametry EA i SA
`solvers/EvolutionaryAlgorithm`	algorytm ewolucyjny
`solvers/SimulatedAnnealing`	symulowane wyżarzanie
`Logger`	zapis przebiegu do CSV
`Stats`	best, worst, avg, std
`main.cpp`	testy, pełne porównanie metod, pojedyncze eksperymenty z argumentów
`scripts/*.py`	wykresy, tabela, serie eksperymentów

4. Dane: Item, City, Problem

```
struct Item {
    int id = 0;
    int profit = 0;
    int weight = 0;
    int city = 0;
};

struct City {
    int x = 0;
    int y = 0;
};

struct Problem {
    std::string name;

    int numCities = 0;
    int numItems = 0;
    int capacity = 0;

    double minSpeed = 0.0;
    double maxSpeed = 0.0;
    double rentingRatio = 0.0;

    std::vector<City> cities;
    std::vector<Item> items;
    std::vector<std::vector<int>> itemsByCity;
};
```

Wyjaśnienie:

- `struct` zamiast `class`, bo to same dane, bez logiki.
- `= 0` przy każdym polu: bez tego pola miałyby losowe wartości ze śmieci w pamięci.
- `int` zamiast `unsigned`: odejmowanie na `unsigned` "zawija się" do ogromnych liczb zamiast dawać wynik ujemny.
- `double` dla prędkości: `float` ma tylko ok. 7 cyfr dokładności, za mało przy sumowaniu setek czasów.
- **Numeracja od 0:** w pliku miasta i przedmioty są od 1, w kodzie od 0. Odejmujemy 1 przy wczytywaniu.
- **`itemsByCity`**: dla każdego miasta lista numerów przedmiotów, które w nim leżą. Przykład:

  ```
  itemsByCity[0] = {}          miasto startowe, pusto
  itemsByCity[1] = {0}         w mieście 1 leży przedmiot 0
  itemsByCity[2] = {1, 280}    w mieście 2 leżą przedmioty 1 i 280
  ```

  Dzięki temu funkcja oceny nie musi przeszukiwać wszystkich przedmiotów przy każdym mieście.

---

5. Loader: wczytywanie pliku .ttp

Format pliku

```
PROBLEM NAME:  a280-TTP                       <- nagłówek: KLUCZ: wartość
KNAPSACK DATA TYPE: uncorrelated
DIMENSION:  280
NUMBER OF ITEMS:  279
CAPACITY OF KNAPSACK:  12718
MIN SPEED:  0.1
MAX SPEED:  1
RENTING RATIO:  5.93
EDGE_WEIGHT_TYPE:  CEIL_2D
NODE_COORD_SECTION  (INDEX, X, Y):            <- znacznik sekcji miast
1  288  149                                   <- miasto: id x y
2  288  129
...
ITEMS SECTION  (INDEX, PROFIT, WEIGHT, ...):  <- znacznik sekcji przedmiotów
1  119  1  2                                  <- przedmiot: id zysk waga miasto
...
```

W prawdziwym pliku wartości są oddzielone tabulatorami, tutaj dla czytelności spacjami.

Pułapki w pliku:

- końce linii są windowsowe (`\r\n`), więc na końcu każdej linii zostaje znak `\r`,
- separatory to mieszanka spacji i tabulatorów,
- słowo `ITEMS` występuje też w `NUMBER OF ITEMS`, dlatego sekcję rozpoznajemy po pełnym `ITEMS SECTION`.

Główna pętla

```
Problem loadProblem(const std::string& path) {
    std::ifstream file(path);
    if (!file.is_open()) {
        throw std::runtime_error("Nie mozna otworzyc pliku: " + path);
    }

    Problem problem;
    Section section = Section::Header;
    std::string line;

    while (std::getline(file, line)) {
        // Usun '\r' z konca (pliki maja windowsowe konce linii)
        if (!line.empty() && line.back() == '\r') {
            line.pop_back();
        }

        // Pomin puste linie
        if (trim(line).empty()) {
            continue;
        }

        // Przelaczanie sekcji (te linie same nie sa danymi)
        if (line.starts_with("NODE_COORD_SECTION")) {
            section = Section::Cities;
            continue;
        }
        if (line.starts_with("ITEMS SECTION")) {
            section = Section::Items;
            continue;
        }

        // Linia z danymi: co z nia zrobic, zalezy od sekcji
        switch (section) {
            case Section::Header:
                parseHeaderLine(line, problem);
                break;
            case Section::Cities:
                parseCityLine(line, problem);
                break;
            case Section::Items:
                parseItemLine(line, problem);
                break;
        }
    }

    validate(problem, path);
    buildItemsByCity(problem);

    return problem;
}
```

Krok po kroku:

1. **Otwarcie pliku.** Jeśli się nie da (zła ścieżka), rzucamy wyjątek z nazwą pliku. Słowo `throw` jest konieczne, bez niego wyjątek tylko by się utworzył i zniknął.

2. `section` pamięta, w której części pliku jesteśmy. Na początku jest to nagłówek.
3. `std::getline` czyta plik linia po linii. Pętla kończy się sama na końcu pliku.
4. **Usuwanie `\r`.** Jeśli ostatni znak linii to `\r`, usuwamy go przez `pop_back()`. Uwaga: `'\r'` to jeden znak, a `'/r'` (ukośnik w drugą stronę) to błąd.
5. **Puste linie** pomijamy przez `continue`.
6. **Znaczniki sekcji** tylko zmieniają `section`. Same nie są danymi, więc po zmianie od razu `continue`.
7. `switch` wybiera, jak przetworzyć linię z danymi. `break` na końcu każdego `case` jest konieczny, bo bez niego wykonanie "przeleciałoby" do następnego przypadku.
8. **Po pętli:** sprawdzamy poprawność (`validate`) i budujemy `itemsByCity`.

Linia nagłówka

```
void parseHeaderLine(const std::string& line, Problem& problem) {
    const auto colon = line.find(':');
    if (colon == std::string::npos) {
        return;
    }

    const std::string key = trim(line.substr(0, colon));
    const std::string value = trim(line.substr(colon + 1));

    if (key == "PROBLEM NAME") {
        problem.name = value;
    } else if (key == "DIMENSION") {
        problem.numCities = std::stoi(value);
    } else if (key == "NUMBER OF ITEMS") {
        problem.numItems = std::stoi(value);
    } else if (key == "CAPACITY OF KNAPSACK") {
        problem.capacity = std::stoi(value);
    } else if (key == "MIN SPEED") {
        problem.minSpeed = std::stod(value);
    } else if (key == "MAX SPEED") {
        problem.maxSpeed = std::stod(value);
    } else if (key == "RENTING RATIO") {
        problem.rentingRatio = std::stod(value);
    }
}
```

Krok po kroku (na przykładzie `"CAPACITY OF KNAPSACK: \t12718"`):

1. `find(':')` znajduje pozycję dwukropka. `npos` oznacza "nie znaleziono", wtedy linia nie jest nagłówkiem.
2. `substr(0, colon)` to tekst przed dwukropkiem: `"CAPACITY OF KNAPSACK"`.
3. `substr(colon + 1)` to tekst po dwukropku: `" \t12718"`.
4. `trim` obcina spacje i tabulatory z obu końców: `"12718"`.
5. Porównujemy klucz z nazwami i zamieniamy tekst na liczbę: `std::stoi` na `int`, `std::stod` na `double`.

Linia miasta i przedmiotu

```
void parseCityLine(const std::string& line, Problem& problem) {
    std::istringstream iss(line);
    int id = 0;
    City city;
    if (!(iss >> id >> city.x >> city.y)) {
        throw std::runtime_error("Bledna linia miasta: " + line);
    }
    problem.cities.push_back(city);
}

void parseItemLine(const std::string& line, Problem& problem) {
    std::istringstream iss(line);
    Item item;
    if (!(iss >> item.id >> item.profit >> item.weight >> item.city)) {
        throw std::runtime_error("Bledna linia przedmiotu: " + line);
    }
    item.id -= 1;
    item.city -= 1;
    problem.items.push_back(item);
}
```

Krok po kroku:

1. `std::istringstream iss(line)` zamienia tekst linii w strumień, z którego czyta się jak z `std::cin`.
2. `iss >> a >> b >> c` wczytuje liczby po kolei, sam pomijając spacje i tabulatory.
3. Jeśli wczytanie się nie udało (np. litery zamiast liczb), `if (!(...))` to wykrywa i rzucamy wyjątek.
4. `id` miasta wczytujemy, ale nie używamy, bo numer miasta to jego pozycja w wektorze. Trzeba go jednak "zjeść", żeby dostać się do `x` i `y`.
5. W przedmiocie odejmujemy 1 od `id` i `city`, bo w kodzie numerujemy od 0.
6. `push_back` dopisuje element na koniec wektora.

Sprawdzanie i itemsByCity

```
void buildItemsByCity(Problem& problem) {
    problem.itemsByCity.assign(problem.numCities, {});
    for (const Item& item : problem.items) {
        problem.itemsByCity[item.city].push_back(item.id);
    }
}
```

1. `assign(numCities, {})` tworzy `numCities` pustych list.
2. Dla każdego przedmiotu dopisujemy jego numer do listy jego miasta.

`validate` sprawdza, czy liczba wczytanych miast i przedmiotów zgadza się z nagłówkiem, czy pojemność jest dodatnia, czy `0 < vmin < vmax` i czy każdy przedmiot leży w istniejącym mieście. Jeśli coś jest nie tak, rzuca wyjątek.

Funkcje pomocnicze są w `namespace { }` (przestrzeń anonimowa), więc są widoczne tylko w `Loader.cpp` i nie kolidują z nazwami w innych plikach.

6. Distance: macierz odległości

```
class Distance {
public:
    explicit Distance(const Problem& problem);
    int get(int from, int to) const;

private:
    int numCities_ = 0;
    std::vector<int> matrix_;  // plaska macierz n*n
};
```

```
Distance::Distance(const Problem& problem)
    : numCities_(problem.numCities),
      matrix_(numCities_ * numCities_, 0) {
    for (int i = 0; i < numCities_; ++i) {
        const City& a = problem.cities[i];

        for (int j = 0; j < numCities_; ++j) {
            const City& b = problem.cities[j];

            const double dx = static_cast<double>(a.x - b.x);
            const double dy = static_cast<double>(a.y - b.y);
            const double euclidean = std::sqrt(dx * dx + dy * dy);

            matrix_[i * numCities_ + j] = static_cast<int>(std::ceil(euclidean));
        }
    }
}

int Distance::get(int from, int to) const {
    return matrix_[from * numCities_ + to];
}
```

Po co: funkcja oceny pyta o odległość setki tysięcy razy. Liczymy wszystko raz i potem tylko odczytujemy.

Krok po kroku:

1. **Lista inicjalizacyjna** (`: numCities_(...), matrix_(...)`) ustawia pola, zanim wykona się ciało konstruktora. `matrix_(n * n, 0)` tworzy od razu wektor `n*n` zer. Kolejność musi zgadzać się z kolejnością pól w klasie, bo `matrix_` korzysta z `numCities_`.
2. **Dwie pętle** przechodzą przez wszystkie pary miast `(i, j)`.
3. **`const City& a`**: referencja, czyli patrzymy na miasto bez kopiowania.
4. **Odległość euklidesowa:** `√(dx² + dy²)` (twierdzenie Pitagorasa). Liczymy na `double`.
5. **`std::ceil`** zaokrągla w górę, bo plik ma `EDGE_WEIGHT_TYPE: CEIL_2D`.
6. **Płaska tablica:** zamiast tabeli 2D trzymamy wiersze jeden za drugim. Element `(i, j)` leży pod indeksem `i * n + j`:

```
tabela 3x3:          płasko:
[a b c]              [a b c d e f g h i]
[d e f]               0 1 2 3 4 5 6 7 8
[g h i]              (1, 2) → 1*3 + 2 = 5 → f
```

   To szybsze niż `vector<vector<int>>`, bo wszystkie dane leżą w pamięci obok siebie.
7. **`get(...) const`**: `const` na końcu oznacza, że metoda nic nie zmienia w obiekcie.

Test: miasta `(288,149)` i `(288,129)` dają `get(0, 1) = 20`.

---

7. Rng: losowość z seedem

```
class Rng {
public:
    explicit Rng(std::uint32_t seed) : seed_(seed), engine_(seed) {}

    int uniformInt(int min, int max) {
        std::uniform_int_distribution<int> dist(min, max);
        return dist(engine_);
    }

    double uniformReal() {
        std::uniform_real_distribution<double> dist(0.0, 1.0);
        return dist(engine_);
    }

    std::mt19937& engine() { return engine_; }

private:
    std::uint32_t seed_;
    std::mt19937 engine_;
};
```

- **`std::mt19937`** to standardowy, szybki generator dobrej jakości.
- **Seed:** ten sam seed zawsze daje te same "losowe" liczby, więc każdy eksperyment da się powtórzyć.
- **`uniformInt(min, max)`**: liczba całkowita z `[min, max]`, oba końce włącznie.
- **`uniformReal()`**: liczba z `[0, 1)`. Przyda się do prawdopodobieństw: `if (rng.uniformReal() < 0.1)` jest prawdziwe w 10% przypadków.
- **Dlaczego nie `rand()`:** na Windowsie `RAND_MAX` to tylko 32767, jakość jest słaba, a `rand() % n` faworyzuje małe liczby.

---

8. Individual: osobnik

```
struct Individual {
    std::vector<int> route;     // kolejnosc miast
    std::vector<char> picked;   // picked[k] == 1: bierzemy przedmiot k

    double fitness = 0.0;       // G
    bool evaluated = false;     // czy fitness jest aktualny

    long long totalProfit = 0;
    int totalWeight = 0;
    double travelTime = 0.0;
};
```

Reprezentacja trasy (jak w treści zadania): ciąg miast, np. `[3, 5, 1, 2, 4]`. Złodziej startuje z `route[0]`, odwiedza resztę po kolei i wraca do startu. Każde miasto występuje dokładnie raz, czyli trasa jest **permutacją**.

`char` zamiast `bool`: `std::vector<bool>` jest w C++ specjalnym przypadkiem (pakuje bity), który działa wolniej i inaczej niż zwykły wektor.

Losowy osobnik

```
Individual makeIdentityIndividual(const Problem& problem) {
    Individual individual;
    individual.route.resize(problem.numCities);
    std::iota(individual.route.begin(), individual.route.end(), 0); // 0, 1, 2, ..., n-
1
    individual.picked.assign(problem.numItems, 0);
    return individual;
}

Individual makeRandomIndividual(const Problem& problem, Rng& rng) {
    Individual individual = makeIdentityIndividual(problem);
    std::shuffle(individual.route.begin(), individual.route.end(), rng.engine());
    return individual;
}
```

1. `std::iota` wypełnia trasę liczbami `0, 1, 2, ..., n-1`.
2. `std::shuffle` tasuje je losowo jak talię kart, używając naszego generatora z seedem.
3. `picked` to same zera. Plan przedmiotów wypełni `PackingHeuristic` przy ocenie.

Sprawdzanie poprawności

```
bool isValidRoute(const Individual& individual, int numCities) {
    if (static_cast<int>(individual.route.size()) != numCities) {
        return false;
    }

    std::vector<char> seen(numCities, 0);
    for (const int city : individual.route) {
        if (city < 0 || city >= numCities || seen[city]) {
            return false; // miasto spoza zakresu albo powtorzone
        }
        seen[city] = 1;
    }
    return true;
}
```

Odhaczamy każde miasto w tablicy `seen`. Jeśli jakieś pojawi się drugi raz albo jest spoza zakresu, trasa jest zła. Używane w testach krzyżowania i mutacji.

---

9. PackingHeuristic: wybór przedmiotów

Osobnik ma tylko trasę. Treść każe zamienić TSP w TTP algorytmem zachłannym, czyli dla danej trasy wybrać przedmioty.

Idea: przedmiot zebrany wcześnie jest niesiony długo, więc długo spowalnia. Opłaca się brać przedmioty z dużym zyskiem, małą wagą i blisko końca trasy.

```
score = profit / (weight * droga_do_konca_trasy)
```

```
void packItems(const Problem& problem, const Distance& distance, Individual&
individual) {
    int n = problem.numCities;
    int m = problem.numItems;

    // 1. Na ktorej pozycji trasy jest kazde miasto?
    std::vector<int> positionInRoute(n);
    for (int pos = 0; pos < n; pos++) {
        int city = individual.route[pos];
        positionInRoute[city] = pos;
    }

    // 2. Ile drogi zostalo od pozycji pos do konca trasy (razem z powrotem do startu)?
    std::vector<double> remainingDistance(n);
    int lastCity = individual.route[n - 1];
    int startCity = individual.route[0];
    remainingDistance[n - 1] = distance.get(lastCity, startCity);
    for (int pos = n - 2; pos >= 0; pos--) {
        int cityHere = individual.route[pos];
        int cityNext = individual.route[pos + 1];
        remainingDistance[pos] = remainingDistance[pos + 1] + distance.get(cityHere,
cityNext);
    }

    // 3. Ocena kazdego przedmiotu. Trzymamy pary (ocena, numer przedmiotu).
    std::vector<std::pair<double, int>> scores;
    for (int k = 0; k < m; k++) {
        const Item& item = problem.items[k];
        int pos = positionInRoute[item.city];

        if (pos == 0) {
            continue; // w miescie startowym nic nie zbieramy
        }

        double dist = remainingDistance[pos];
        if (dist < 1.0) {
            dist = 1.0; // zabezpieczenie przed dzieleniem przez 0
        }

        double score = item.profit / (item.weight * dist);
        scores.push_back(std::make_pair(score, k));
    }

    // 4. Sortujemy rosnaco po ocenie, wiec najlepsze sa na koncu.
    std::sort(scores.begin(), scores.end());

    // 5. Bierzemy od najlepszego (od konca), jesli sie miesci.
    individual.picked.assign(m, 0);
    int currentWeight = 0;
    for (int i = static_cast<int>(scores.size()) - 1; i >= 0; i--) {
        int k = scores[i].second;
        int weight = problem.items[k].weight;

        if (currentWeight + weight <= problem.capacity) {
            individual.picked[k] = 1;
            currentWeight += weight;
        }
```

```
    }
}
```

Krok po kroku na przykładzie trasy `0 → 1 → 2 → 3 → 0` z `tests/tiny.ttp`:

1. **Pozycje miast w trasie.** `positionInRoute[miasto] = pozycja`. Trasa `[0, 1, 2, 3]` daje `positionInRoute = [0, 1, 2, 3]`. Dla trasy `[2, 0, 3, 1]` byłoby `positionInRoute[2] = 0`, `positionInRoute[0] = 1` itd.

2. **Droga do końca.** Idziemy od końca trasy i sumujemy odcinki:

   ```
   pozycja 3 (miasto 3):  3→0          = 4
   pozycja 2 (miasto 2):  2→3 + 4      = 3 + 4  = 7
   pozycja 1 (miasto 1):  1→2 + 7      = 4 + 7  = 11
   pozycja 0 (miasto 0):  0→1 + 11     = 3 + 11 = 14
   ```

3. **Ocena przedmiotów:**

   ```
   p0 (miasto 1): 10 / (2 × 11) = 0.45
   p1 (miasto 2): 20 / (3 × 7)  = 0.95
   p2 (miasto 3):  5 / (4 × 4)  = 0.31
   ```

   Przedmioty z miasta startowego pomijamy (`pos == 0`). Pary `(ocena, numer)` trzymamy razem, żeby po sortowaniu wiedzieć, który przedmiot ma jaką ocenę.

4. **Sortowanie.** `std::sort` na parach sortuje rosnąco po pierwszym elemencie (ocenie): `[(0.31, p2), (0.45, p0), (0.95, p1)]`.

5. **Pakowanie od najlepszego** (pętla od końca listy):

   ```
   p1, waga 3:  0 + 3 = 3 ≤ 5  → bierzemy
   p0, waga 2:  3 + 2 = 5 ≤ 5  → bierzemy
   p2, waga 4:  5 + 4 = 9 > 5  → nie mieści się
   ```

   Wynik: `picked = [1, 1, 0]`.

Ograniczenie: heurystyka bierze wszystko, co się zmieści, nawet jeśli spowolnienie kosztuje więcej, niż przedmiot daje zysku. W `tiny.ttp` pusty plecak dałby `G = -14`, a heurystyka daje `G = -49.25`. Próba ulepszenia (pakowanie "smart") jest opisana w sekcji 22.

Funkcja ma dodatkowy parametr `bool profitCheck = false`. Domyślnie działa dokładnie tak, jak opisano wyżej.

---

10. Evaluator: funkcja oceny

```
class Evaluator {
public:
    Evaluator(const Problem& problem, const Distance& distance, bool smartPacking =
false);
    double evaluate(Individual& individual);
    long long getCount() const;
    void resetCount();

private:
    const Problem& problem_;
    const Distance& distance_;
    bool smartPacking_ = false;
    long long count_ = 0;
};
```

Klasa trzyma **referencje** do problemu i odległości (bez kopiowania) oraz **licznik ocen**. Treść każe porównywać metody po liczbie wywołań funkcji oceny.

```
double Evaluator::evaluate(Individual& individual) {
    count_++;

    // Najpierw wybieramy przedmioty dla tej trasy
    packItems(problem_, distance_, individual, smartPacking_);

    int n = problem_.numCities;
    double vmax = problem_.maxSpeed;
    double vmin = problem_.minSpeed;
    double W = problem_.capacity;

    int currentWeight = 0;    // Wc
    long long profit = 0;     // g(y)
    double time = 0.0;        // f(x, y)

    for (int pos = 1; pos < n; pos++) {
        int from = individual.route[pos - 1];
        int to = individual.route[pos];

        // Predkosc zalezy od tego, ile niesiemy
        double v = vmax - currentWeight * (vmax - vmin) / W;
        time += distance_.get(from, to) / v;

        // Po dojechaniu zbieramy wybrane przedmioty z tego miasta
        for (int k : problem_.itemsByCity[to]) {
            if (individual.picked[k] == 1) {
                currentWeight += problem_.items[k].weight;
                profit += problem_.items[k].profit;
            }
        }
    }

    // Powrot z ostatniego miasta do startowego (bez zbierania)
    int lastCity = individual.route[n - 1];
    int startCity = individual.route[0];
    double v = vmax - currentWeight * (vmax - vmin) / W;
    time += distance_.get(lastCity, startCity) / v;

    double G = profit - time;

    individual.fitness = G;
    individual.evaluated = true;
    individual.totalProfit = profit;
    individual.totalWeight = currentWeight;
    individual.travelTime = time;

    return G;
}
```

Krok po kroku:

1. `count_++`: liczymy każde wywołanie.
2. `packItems` ustala plan przedmiotów dla tej trasy.
3. **Pętla po trasie** od pozycji 1: jedziemy z `route[pos - 1]` do `route[pos]`.
   - prędkość liczymy z wagi, którą niesiemy **przed** dojechaniem do miasta,
   - czas odcinka = odległość / prędkość,
   - **po** dojechaniu podnosimy wybrane przedmioty z tego miasta (`itemsByCity[to]`).

4. **Powrót do startu** z aktualną (pełną) wagą, bez zbierania.
5. `G = zysk - czas`, zapis wyników w osobniku, `evaluated = true`.

Przykład (`tests/tiny.ttp`, `W = 5`, `vmin = 0.1`, `vmax = 1`, `picked = [1, 1, 0]`):

Odcinek	Wc przed	v	Czas	Po dojechaniu
0 → 1	0	1.00	3 / 1.00 = 3.00	bierze p0: Wc = 2, zysk = 10
1 → 2	2	0.64	4 / 0.64 = 6.25	bierze p1: Wc = 5, zysk = 30
2 → 3	5	0.10	3 / 0.10 = 30.00	p2 niewybrany
3 → 0	5	0.10	4 / 0.10 = 40.00	powrót
			razem 79.25	

```
G = 30 - 79.25 = -49.25
```

Program zwraca dokładnie tę wartość (test w `main.cpp`). Pełne obliczenia są w `tests/tiny_expected.md`.

---

11. Algorytm losowy

```
SolverResult runRandomSearch(const Problem& problem, Evaluator& evaluator, Rng& rng,
int count) {
    SolverResult result;

    for (int i = 0; i < count; i++) {
        Individual individual = makeRandomIndividual(problem, rng);
        double G = evaluator.evaluate(individual);
        result.values.push_back(G);

        if (i == 0 || G > result.best.fitness) {
            result.best = individual;
        }
    }

    return result;
}
```

1. `count` razy (w treści 10 000) losujemy trasę i ją oceniamy.
2. Każde `G` dopisujemy do `values`, z których potem liczymy statystyki.
3. Zapamiętujemy najlepszego osobnika. Warunek `i == 0` łapie pierwszego, bo na początku nie ma z kim porównać.

`SolverResult` to wspólny wynik każdej metody: lista wszystkich `G` i najlepszy osobnik.

---

12. Algorytm zachłanny

Najbliższy sąsiad

```
Individual makeNearestNeighbourIndividual(const Problem& problem, const Distance&
distance, int startCity) {
    int n = problem.numCities;

    Individual individual;
    individual.picked.assign(problem.numItems, 0);

    std::vector<bool> visited(n, false);
    int current = startCity;
    individual.route.push_back(current);
    visited[current] = true;

    for (int step = 1; step < n; step++) {
        int nearest = -1;
        int nearestDistance = 0;

        for (int city = 0; city < n; city++) {
            if (visited[city]) {
                continue;
            }
            int d = distance.get(current, city);
            if (nearest == -1 || d < nearestDistance) {
                nearest = city;
                nearestDistance = d;
            }
        }

        individual.route.push_back(nearest);
        visited[nearest] = true;
        current = nearest;
    }

    return individual;
}
```

Krok po kroku:

1. Zaczynamy w `startCity` i oznaczamy je jako odwiedzone.
2. `n - 1` razy:
   - przeglądamy wszystkie nieodwiedzone miasta i szukamy najbliższego od `current`. `nearest == -1` oznacza "jeszcze nic nie znaleziono", więc pierwsze nieodwiedzone miasto zawsze staje się kandydatem,
   - dopisujemy je do trasy, oznaczamy jako odwiedzone i stajemy się w nim.

Przykład z `tiny.ttp`, start z 0:

```
z 0: najbliżej 1 (odległość 3)   → [0, 1]
z 1: najbliżej 2 (odległość 4)   → [0, 1, 2]
z 2: zostało tylko 3             → [0, 1, 2, 3]
```

Uruchomienie z każdego miasta

```
SolverResult runGreedy(const Problem& problem, const Distance& distance, Evaluator&
evaluator) {
    SolverResult result;

    for (int start = 0; start < problem.numCities; start++) {
        Individual individual = makeNearestNeighbourIndividual(problem, distance,
start);
        double G = evaluator.evaluate(individual);
        result.values.push_back(G);

        if (start == 0 || G > result.best.fitness) {
            result.best = individual;
        }
    }

    return result;
}
```

Najbliższy sąsiad jest deterministyczny: z tego samego startu zawsze daje tę samą trasę. Dlatego, zgodnie z treścią, uruchamiamy go z **każdego** miasta startowego (280 uruchomień) i liczymy statystyki z tych wyników.

---

13. Mutacje: swap i inwersja

Funkcje pomocnicze

```
void swapCities(std::vector<int>& route, int i, int j) {
    int temp = route[i];
    route[i] = route[j];
    route[j] = temp;
}

void reverseSegment(std::vector<int>& route, int i, int j) {
    while (i < j) {
        swapCities(route, i, j);
        i++;
        j--;
    }
}
```

- **`swapCities`**: klasyczna zamiana przez zmienną pomocniczą.
- **`reverseSegment`**: odwraca fragment `[i, j]`, zamieniając skrajne elementy i idąc do środka:

  ```
  [6 7 1 2 3]  zamień 6↔3 → [3 7 1 2 6]
               zamień 7↔2 → [3 2 1 7 6]
               i == j, koniec
  ```

Swap (Pm na gen)

```
void mutateSwap(Individual& individual, double pm, Rng& rng) {
    int n = static_cast<int>(individual.route.size());
    bool changed = false;

    for (int i = 0; i < n; i++) {
        if (rng.uniformReal() < pm) {
            int j = rng.uniformInt(0, n - 1);
            swapCities(individual.route, i, j);
            changed = true;
        }
    }

    if (changed) {
        individual.evaluated = false;
    }
}
```

1. Dla **każdej pozycji** losujemy liczbę z `[0, 1)`. Jeśli jest mniejsza od `pm`, zamieniamy to miasto z losowym innym.
2. Przy `pm = 0.01` i 280 miastach średnio zachodzi ok. 2.8 zamian na osobnika.
3. Jeśli trasa się zmieniła, `evaluated = false`, bo stara ocena jest nieaktualna.

Przykład z treści: `5671234 → 5371264` (zamiana pozycji 1 i 5).

Inwersja (Pm na osobnika)

```
void mutateInversion(Individual& individual, double pm, Rng& rng) {
    if (rng.uniformReal() >= pm) {
        return; // tym razem bez mutacji
    }

    int n = static_cast<int>(individual.route.size());
    int i = rng.uniformInt(0, n - 1);
    int j = rng.uniformInt(0, n - 1);

    if (i > j) {
        int temp = i;
        i = j;
        j = temp;
    }

    reverseSegment(individual.route, i, j);
    individual.evaluated = false;
}
```

1. **Raz na osobnika** rzucamy monetą z szansą `pm`. Przy `pm = 0.1` mutuje średnio 10% populacji.
2. Losujemy dwa punkty i ustawiamy je tak, żeby `i ≤ j`.
3. Odwracamy fragment `[i, j]`.

Przykład z treści: `5[67123]4 → 5[32176]4`.

Różnica w Pm: swap zmienia tylko 2 geny, więc `Pm` liczymy na gen (start od 0.01). Inwersja zmienia cały fragment, więc `Pm` liczymy na osobnika (start od 0.1). Tak sugeruje treść.

---

14. Krzyżowanie: OX i PMX

Problem przy krzyżowaniu permutacji: zwykłe "pół od jednego rodzica, pół od drugiego" daje duplikaty i braki miast. OX i PMX zawsze dają poprawną permutację.

Każdy operator zwraca **jedno dziecko**. Drugie dziecko dostajemy, wywołując operator z zamienionymi rodzicami.

OX (Order Crossover)

```
std::vector<int> orderCrossover(const std::vector<int>& parent1, const
std::vector<int>& parent2, int a, int b) {
    int n = static_cast<int>(parent1.size());

    std::vector<int> child(n, -1);         // -1 = puste miejsce
    std::vector<bool> used(n, false);      // used[miasto] = czy miasto juz jest w
dziecku

    // 1. Kopiujemy fragment [a, b] z rodzica 1
    for (int i = a; i <= b; i++) {
        child[i] = parent1[i];
        used[parent1[i]] = true;
    }

    // 2. Wypelniamy puste miejsca miastami z rodzica 2 (w jego kolejnosci)
    int pos = 0;
    for (int k = 0; k < n; k++) {
        int city = parent2[k];
        if (used[city]) {
            continue;
        }

        while (child[pos] != -1) {
            pos++;
        }

        child[pos] = city;
        used[city] = true;
    }

    return child;
}
```

Krok po kroku (fragment `[3, 5]`):

```
Rodzic 1:  0 1 2 [3 4 5] 6 7 8
Rodzic 2:  8 2 6  7 1 5  4 0 3

Krok 1, fragment z rodzica 1:
Dziecko:   _ _ _ [3 4 5] _ _ _

Krok 2, idziemy po rodzicu 2 i wstawiamy brakujące miasta w wolne miejsca od lewej:
8 → wolne    → 8 _ _ [3 4 5] _ _ _
2 → wolne    → 8 2 _ [3 4 5] _ _ _
6 → wolne    → 8 2 6 [3 4 5] _ _ _
7 → wolne    → 8 2 6 [3 4 5] 7 _ _   (pos przeskakuje fragment)
1 → wolne    → 8 2 6 [3 4 5] 7 1 _
5 → już jest, pomijamy
4 → już jest, pomijamy
0 → wolne    → 8 2 6 [3 4 5] 7 1 0
3 → już jest, pomijamy

Dziecko:   8 2 6 3 4 5 7 1 0
```

Dziecko dziedziczy **fragment** z rodzica 1 i **kolejność** pozostałych miast z rodzica 2.

PMX (Partially Mapped Crossover)

```
std::vector<int> pmxCrossover(const std::vector<int>& parent1, const std::vector<int>&
parent2, int a, int b) {
    int n = static_cast<int>(parent1.size());

    std::vector<int> child(n, -1);
    std::vector<bool> inSegment(n, false);
    std::vector<int> positionInParent1(n);

    for (int i = 0; i < n; i++) {
        positionInParent1[parent1[i]] = i;
    }

    // 1. Kopiujemy fragment [a, b] z rodzica 1
    for (int i = a; i <= b; i++) {
        child[i] = parent1[i];
        inSegment[parent1[i]] = true;
    }

    // 2. Reszta z rodzica 2, z poprawkami przez mapowanie
    for (int i = 0; i < n; i++) {
        if (i >= a && i <= b) {
            continue;
        }

        int city = parent2[i];

        while (inSegment[city]) {
            int pos = positionInParent1[city];
            city = parent2[pos];
        }

        child[i] = city;
    }

    return child;
}
```

Krok po kroku (ci sami rodzice, fragment `[3, 5]`):

```
Rodzic 1:  0 1 2 [3 4 5] 6 7 8
Rodzic 2:  8 2 6 [7 1 5] 4 0 3

Pary z fragmentu (rodzic 1 ↔ rodzic 2 na tych samych pozycjach):
3 ↔ 7,  4 ↔ 1,  5 ↔ 5

Krok 1, fragment z rodzica 1:
Dziecko:   _ _ _ [3 4 5] _ _ _

Krok 2, pozostałe pozycje z rodzica 2 na tych samych miejscach:
poz. 0: 8 → nie ma we fragmencie → 8
poz. 1: 2 → nie ma               → 2
poz. 2: 6 → nie ma               → 6
poz. 6: 4 → JEST we fragmencie!
        4 stoi w rodzicu 1 na poz. 4, w rodzicu 2 na poz. 4 jest 1
        1 nie ma we fragmencie   → 1
poz. 7: 0 → nie ma               → 0
poz. 8: 3 → JEST we fragmencie!
        3 stoi w rodzicu 1 na poz. 3, w rodzicu 2 na poz. 3 jest 7
        7 nie ma we fragmencie   → 7

Dziecko:   8 2 6 3 4 5 1 0 7
```

Pętla `while` jest potrzebna, bo mapowanie może prowadzić łańcuchem (A→B, B→C). Idziemy po nim, aż trafimy na miasto spoza fragmentu.

Dziecko dziedziczy fragment z rodzica 1 i **pozycje** pozostałych miast z rodzica 2 (tam, gdzie się da).

Wersje dla osobników

```
Individual crossoverOX(const Individual& parent1, const Individual& parent2, Rng& rng)
{
    int n = static_cast<int>(parent1.route.size());
    int a = 0;
    int b = 0;
    randomCutPoints(n, rng, a, b);

    Individual child;
    child.route = orderCrossover(parent1.route, parent2.route, a, b);
    child.picked.assign(parent1.picked.size(), 0);
    child.evaluated = false;
    return child;
}
```

Losują punkty cięcia `a ≤ b`, wołają wersję z punktami i zwracają nowego, jeszcze nieocenionego osobnika. Plan przedmiotów policzy `Evaluator`.

Dwie wersje (z podanymi punktami i z losowymi) są po to, żeby dało się przetestować operator na znanym przykładzie, a w algorytmie używać losowych punktów.

---

15. Statystyki

```
Stats computeStats(const std::vector<double>& values) {
    Stats stats;
    if (values.empty()) {
        return stats;
    }

    stats.best = values[0];
    stats.worst = values[0];
    double sum = 0.0;
    for (double value : values) {
        if (value > stats.best) {
            stats.best = value;
        }
        if (value < stats.worst) {
            stats.worst = value;
        }
        sum += value;
    }
    stats.avg = sum / values.size();

    double sumSquares = 0.0;
    for (double value : values) {
        double diff = value - stats.avg;
        sumSquares += diff * diff;
    }
    stats.std = std::sqrt(sumSquares / values.size());

    return stats;
}
```

1. **Pierwsza pętla:** największa wartość (`best`, bo maksymalizujemy `G`), najmniejsza (`worst`) i suma.
2. **Średnia** = suma / liczba wartości.
3. **Druga pętla:** odchylenie standardowe `std = √(średnia z (wartość - średnia)²)`. Mówi, jak bardzo wyniki rozrzucają się wokół średniej.

To dokładnie kolumny best / worst / avg / std z Tabeli 1 w treści zadania.

---

16. Config: parametry algorytmów

```
enum class SelectionType { Tournament, Roulette };
enum class CrossoverType { OX, PMX };
enum class MutationType  { Swap, Inversion };

struct EAConfig {
    int popSize = 100;
    int generations = 100;
    double px = 0.7;          // prawdopodobieństwo krzyżowania
    double pm = 0.1;          // prawdopodobieństwo mutacji
    int tourSize = 5;         // rozmiar turnieju
    int elitism = 1;          // ilu najlepszych przechodzi bez zmian
    int maxEvaluations = 0;   // limit ocen (0 = brak limitu)
    double greedyInit = 0.0;  // część populacji startowej z algorytmu zachłannego

    SelectionType selection = SelectionType::Tournament;
    CrossoverType crossover = CrossoverType::OX;
    MutationType mutation = MutationType::Inversion;
};

struct SAConfig {
    int iterations = 10000;
    double startTemperature = 0.0;  // 0 = dobierz automatycznie
    double startAcceptance = 0.5;   // dla automatycznego T0
    double coolingRate = 0.999;     // T = T * coolingRate po każdej iteracji
    MutationType move = MutationType::Inversion;
    bool greedyStart = false;       // start z trasy zachłannej
    int logEvery = 100;             // co ile iteracji zapis do CSV
};
```

Po co:

- wszystkie parametry są w jednym miejscu, a wartości domyślne to parametry startowe z treści (`pop=100, gen=100, Px=0.7, Pm=0.1, Tour=5`),
- `enum class` zamiast tekstu (`"ox"` , `"pmx"`): kompilator pilnuje, że nie da się wpisać nieistniejącego operatora, a `switch` / `if` po enumie jest szybki,
- `describeConfig(config)` zamienia konfigurację na tekst, np. `pop=100 gen=100 Px=0.7 ... mut=swap` . Ten tekst trafia do CSV obok każdego wyniku, więc przy każdej liczbie wiadomo, jaką konfiguracją została uzyskana (wymóg z treści).

---

17. Selekcja: turniej i ruletka

Obie funkcje zwracają **indeks** wybranego osobnika, a nie jego kopię. Kopiowanie osobnika (trasa 280 liczb + plan 2790 przedmiotów) przy każdym wyborze byłoby niepotrzebnie drogie (o to pyta treść).

Turniej

```
int tournamentSelection(const std::vector<Individual>& population, int tourSize, Rng&
rng) {
    int size = static_cast<int>(population.size());

    // Pierwszy uczestnik turnieju
    int bestIndex = rng.uniformInt(0, size - 1);

    // Pozostali uczestnicy: jeśli któryś jest lepszy, zostaje zwycięzcą
    for (int t = 1; t < tourSize; t++) {
        int index = rng.uniformInt(0, size - 1);
        if (population[index].fitness > population[bestIndex].fitness) {
            bestIndex = index;
        }
    }

    return bestIndex;
}
```

1. Losujemy pierwszego uczestnika i uznajemy go za tymczasowego zwycięzcę.
2. Losujemy kolejnych `tourSize - 1` uczestników. Lepszy od zwycięzcy zajmuje jego miejsce.
3. Zwracamy indeks zwycięzcy.

Presja selekcyjna: `Tour = 1` to wybór całkowicie losowy (brak presji). Im większy `Tour`, tym częściej wygrywają najlepsi. Przy `Tour = pop_size` prawie zawsze wygrywa najlepszy osobnik.

Nie potrzeba żadnej dodatkowej kolekcji (sortowania, kopiowania): wystarczą losowe indeksy.

Ruletka

```
int rouletteSelection(const std::vector<Individual>& population, Rng& rng) {
    int size = static_cast<int>(population.size());

    // 1. Najgorsze G w populacji
    double worst = population[0].fitness;
    for (int i = 1; i < size; i++) {
        if (population[i].fitness < worst) {
            worst = population[i].fitness;
        }
    }

    // 2. Waga osobnika = jego G - najgorsze G (zawsze >= 0)
    double total = 0.0;
    for (int i = 0; i < size; i++) {
        total += population[i].fitness - worst;
    }

    if (total <= 0.0) {
        return rng.uniformInt(0, size - 1);  // wszyscy równi
    }

    // 3. Kręcimy kołem
    double point = rng.uniformReal() * total;
    double sum = 0.0;
    for (int i = 0; i < size; i++) {
        sum += population[i].fitness - worst;
        if (sum >= point) {
            return i;
        }
    }

    return size - 1;  // na wypadek błędów zaokrągleń
}
```

Krok po kroku:

1. **Problem:** w ruletce szansa jest proporcjonalna do przystosowania, a `G` w TTP bywa ujemne (losowe trasy). Ujemnej szansy nie ma, więc od każdego `G` odejmujemy najgorsze `G` w populacji. Najgorszy osobnik dostaje wagę 0.
2. Sumujemy wagi: to "obwód koła".
3. Losujemy punkt na kole i idziemy po osobnikach, dodając ich wagi, aż suma przekroczy punkt. Osobnik z dużą wagą zajmuje duży kawałek koła, więc trafia się częściej.

Przykład: wagi `[0, 10, 30]`, suma 40. Punkt 25 wypada na trzecim osobniku (sumy: 0, 10, 40).

Słabość w TTP: przy `G` rzędu setek tysięcy różnice między osobnikami to ułamki procenta, więc wszyscy mają prawie równe szanse. W badaniach ruletka przegrywa z turniejem na każdej instancji.

---

18. Algorytm ewolucyjny

Funkcje pomocnicze

```
static int selectParent(const std::vector<Individual>& population, const EAConfig&
config, Rng& rng) {
    if (config.selection == SelectionType::Roulette) {
        return rouletteSelection(population, rng);
    }
    return tournamentSelection(population, config.tourSize, rng);
}
```

Analogicznie `makeChild` wybiera OX albo PMX, a `mutate` swap albo inwersję, według konfiguracji. Dzięki temu główna pętla nie zawiera `if`-ów od typów operatorów.

`computeGenerationStats` liczy best/avg/worst populacji (do logów), a `isBetter(a, b)` zwraca `a.fitness > b.fitness` i służy do sortowania przy elitaryzmie.

Główna funkcja

```
EAResult runEvolutionaryAlgorithm(const Problem& problem, const Distance& distance,
Evaluator& evaluator,
                                  const EAConfig& config, Rng& rng) {
    EAResult result;
    long long evaluationsAtStart = evaluator.getCount();

    // 1. Inicjalizacja: część populacji zachłanna (greedyInit), reszta losowa
    int greedyCount = static_cast<int>(config.greedyInit * config.popSize);

    std::vector<Individual> population;
    for (int i = 0; i < config.popSize; i++) {
        Individual individual;
        if (i < greedyCount) {
            int startCity = rng.uniformInt(0, problem.numCities - 1);
            individual = makeNearestNeighbourIndividual(problem, distance, startCity);
        } else {
            individual = makeRandomIndividual(problem, rng);
        }
        evaluator.evaluate(individual);
        population.push_back(individual);

        if (i == 0 || individual.fitness > result.best.fitness) {
            result.best = individual;
        }
    }
    result.history.push_back(computeGenerationStats(population, 0));

    // 2. Kolejne pokolenia
    for (int generation = 1; generation <= config.generations; generation++) {
        // Limit ocen: kończymy, gdy budżet się wyczerpał
        long long used = evaluator.getCount() - evaluationsAtStart;
        if (config.maxEvaluations > 0 && used >= config.maxEvaluations) {
            break;
        }

        std::vector<Individual> newPopulation;

        // 2a. Elitaryzm: najlepsi przechodzą bez zmian
        if (config.elitism > 0) {
            std::vector<Individual> sorted = population;
            std::sort(sorted.begin(), sorted.end(), isBetter);
            for (int e = 0; e < config.elitism && e < config.popSize; e++) {
                newPopulation.push_back(sorted[e]);
            }
        }

        // 2b. Reszta populacji: selekcja -> krzyżowanie -> mutacja -> ocena
        while (static_cast<int>(newPopulation.size()) < config.popSize) {
            int index1 = selectParent(population, config, rng);
            int index2 = selectParent(population, config, rng);

            Individual child;
            if (rng.uniformReal() < config.px) {
                child = makeChild(population[index1], population[index2], config, rng);
            } else {
                child = population[index1];  // bez krzyżowania: kopia rodzica (z jego
oceną)
```

```
            }

            mutate(child, config, rng);

            if (!child.evaluated) {
                evaluator.evaluate(child);
            }

            if (child.fitness > result.best.fitness) {
                result.best = child;
            }

            newPopulation.push_back(child);
        }

        population = newPopulation;
        result.history.push_back(computeGenerationStats(population, generation));
    }

    return result;
}
```

Krok po kroku:

1. **Inicjalizacja.** Tworzymy `popSize` osobników i każdego oceniamy. Pierwsze `greedyInit × popSize` osobników to trasy najbliższego sąsiada z losowych miast startowych (różne miasta dają różne trasy, więc populacja nie jest jednolita), reszta to trasy losowe. Zapisujemy statystyki pokolenia 0.

2. **Limit ocen.** Przed każdym pokoleniem sprawdzamy, ile ocen zużył ten przebieg (`evaluator.getCount() - evaluationsAtStart`). Po przekroczeniu `maxEvaluations` kończymy. Dzięki temu EA, SA i losowy mogą mieć ten sam budżet ocen (uczciwe porównanie).

3. **Elitaryzm.** Kopiujemy populację, sortujemy od najlepszego i przenosimy `elitism` najlepszych bez zmian. Najlepsze rozwiązanie nigdy nie ginie.

4. **Tworzenie dzieci**, aż nowa populacja będzie pełna:

   - selekcja dwóch rodziców (indeksy),
   - z szansą `Px` krzyżowanie, w przeciwnym razie dziecko = **kopia** rodzica 1,
   - mutacja (swap albo inwersja, z prawdopodobieństwem `Pm`),
   - ocena **tylko jeśli** osobnik się zmienił. Kopia bez mutacji ma `evaluated == true` i zachowuje ocenę rodzica, więc nie zużywa budżetu,
   - aktualizacja najlepszego osobnika z całego przebiegu.

5. **Wymiana pokoleń:** nowa populacja zastępuje starą, zapisujemy best/avg/worst.

Kopia głęboka czy płytka? (pytanie z treści) `child = population[index1]` kopiuje cały `std::vector` (kopia głęboka). Jest konieczna: dziecko jest potem mutowane, a zmiana nie może zepsuć rodzica, który może być jeszcze wybrany do kolejnego krzyżowania. Rodziców nie kopiujemy, bo selekcja zwraca indeksy.

`EAResult` zawiera najlepszego osobnika i `history`, czyli statystyki każdego pokolenia do wykresów.

---

19. Symulowane wyżarzanie (SA)

Idea: jedno rozwiązanie, które krok po kroku zmieniamy na sąsiada. Lepszego sąsiada zawsze akceptujemy, gorszego czasem też, żeby nie utknąć w lokalnym optimum. Z czasem "temperatura" spada i gorszych ruchów akceptujemy coraz mniej.

Sąsiad

```
static void makeNeighbour(Individual& individual, MutationType move, Rng& rng) {
    int n = static_cast<int>(individual.route.size());
    int i = rng.uniformInt(0, n - 1);
    int j = rng.uniformInt(0, n - 1);

    if (move == MutationType::Swap) {
        swapCities(individual.route, i, j);
    } else {
        if (i > j) {
            int temp = i;
            i = j;
            j = temp;
        }
        reverseSegment(individual.route, i, j);
    }

    individual.evaluated = false;
}
```

Sąsiad to trasa po **jednym** ruchu: zamianie dwóch miast albo odwróceniu fragmentu. To te same funkcje, których używają mutacje.

Automatyczna temperatura początkowa

```
static double estimateStartTemperature(const Individual& start, Evaluator& evaluator,
MutationType move,
                                       double acceptance, Rng& rng) {
    double sumWorse = 0.0;
    int countWorse = 0;
    for (int k = 0; k < 100; k++) {
        Individual neighbour = start;
        makeNeighbour(neighbour, move, rng);
        evaluator.evaluate(neighbour);

        double delta = neighbour.fitness - start.fitness;
        if (delta < 0) {
            sumWorse += -delta;
            countWorse++;
        }
    }

    if (countWorse == 0) {
        return 1.0;
    }
    double averageWorse = sumWorse / countWorse;
    return averageWorse / -std::log(acceptance);
}
```

Problem: `G` ma zupełnie inną skalę w łatwych (dziesiątki tysięcy) i trudnych instancjach (miliony), więc jedna stała `T0` nie pasuje wszędzie.

Rozwiązanie: robimy 100 próbnych ruchów od rozwiązania startowego i liczymy średnie pogorszenie `Δ`. Temperaturę dobieramy tak, żeby taki typowy gorszy ruch był na starcie akceptowany z prawdopodobieństwem `acceptance`:

```
exp(-Δ / T0) = acceptance   →   T0 = Δ / (-ln(acceptance))
```

Dla `acceptance = 0.5`: `T0 = Δ / 0.693`. Te 100 ocen wlicza się do licznika.

Uwaga: przy starcie z trasy zachłannej automatyczne `T0` okazało się za wysokie (SA na początku niszczy dobrą trasę). Dla wersji dostrojonej `T0 = 10` jest podane ręcznie.

Główna pętla

```
    for (int iteration = 1; iteration <= config.iterations; iteration++) {
        Individual neighbour = current;
        makeNeighbour(neighbour, config.move, rng);
        evaluator.evaluate(neighbour);

        double delta = neighbour.fitness - current.fitness;  // > 0 oznacza poprawę
        if (delta >= 0) {
            current = neighbour;
        } else {
            double acceptProbability = std::exp(delta / temperature);
            if (rng.uniformReal() < acceptProbability) {
                current = neighbour;
            }
        }

        if (current.fitness > result.best.fitness) {
            result.best = current;
        }

        temperature = temperature * config.coolingRate;

        if (iteration % config.logEvery == 0) {
            result.history.push_back({iteration, result.best.fitness, current.fitness,
temperature});
        }
    }
```

Krok po kroku:

1. **Start** (przed pętlą): trasa losowa albo zachłanna (`greedyStart`), ocena, `T0` podane albo dobrane automatycznie.
2. **Sąsiad:** kopia aktualnej trasy + jeden ruch, ocena.
3. **Akceptacja** (kryterium Metropolisa):
   - `delta ≥ 0` (lepszy lub równy) → zawsze przechodzimy,
   - `delta < 0` (gorszy) → przechodzimy z prawdopodobieństwem `exp(delta / T)`. Ponieważ `delta` jest ujemne, wynik jest między 0 a 1. Mały spadek i wysoka temperatura dają szansę bliską 1, duży spadek albo niska temperatura dają szansę bliską 0.
4. **Najlepszy z przebiegu:** SA może zejść na gorsze rozwiązanie, więc najlepsze trzymamy osobno.
5. **Schładzanie geometryczne:** `T = T × 0.999`. Po 10 000 iteracjach `T` spada do ok. `0.00005 × T0`.
6. **Historia:** co 100 iteracji zapisujemy najlepszy wynik, bieżący wynik i temperaturę.

Przykład akceptacji: `delta = -50`, `T = 100` → `exp(-0.5) = 0.61`, czyli 61% szans. Przy `T = 10` → `exp(-5) = 0.007`, czyli prawie nigdy.

---

20. Logowanie i wykresy

Logger (C++)

```
void saveHistoryCsv(const std::string& path, const std::vector<GenerationStats>&
history) {
    std::filesystem::path folder = std::filesystem::path(path).parent_path();
    if (!folder.empty()) {
        std::filesystem::create_directories(folder);
    }

    std::ofstream file(path);
    if (!file.is_open()) {
        throw std::runtime_error("Nie mozna utworzyc pliku: " + path);
    }

    file << std::fixed << std::setprecision(2);
    file << "generation;best;avg;worst\n";
    for (const GenerationStats& stats : history) {
        file << stats.generation << ';' << stats.best << ';' << stats.avg << ';' <<
stats.worst << '\n';
    }
}
```

1. `create_directories` tworzy brakujące foldery (np. `results/history`).
2. Każde pokolenie to jeden wiersz `pokolenie;best;avg;worst`, czyli format z treści.

`saveSAHistoryCsv` robi to samo dla SA: `iteration;best;current;temperature`.

Każde uruchomienie ma osobny plik, np. `results/history/a280_n279_uncorr_01_run01.csv` (`historyPath` w `main.cpp` buduje tę nazwę i dokleja zero do numerów 1-9, żeby pliki sortowały się poprawnie).

Skrypty Python

Skrypt	Co robi
`scripts/plot_generations.py`	czyta pliki historii, **uśrednia 10 uruchomień** dla każdego pokolenia/iteracji (`groupby(...).mean()`), rysuje best/avg/worst (EA) albo best/bieżący (SA) i przerywaną linię najlepszego wyniku zachłannego; zapisuje `results/plots/ea_*.png` i `sa_*.png`
`scripts/make_table.py`	z `results/wyniki.csv` robi `results/tabela.md` (tabele dla każdej metody + podsumowanie średnich) i `results/tabela.csv` w układzie Tabeli 1 z treści, z przecinkiem dziesiętnym do polskiego Excela
`scripts/run_experiments.py`	serie eksperymentów do strojenia i badań (sekcja 22)

Ścieżki w skryptach liczone są od położenia skryptu (`Path(__file__).resolve().parent.parent`), więc działają niezależnie od folderu, z którego są uruchomione.

21. main.cpp: dwa tryby pracy

```
int main(int argc, char* argv[]) {
    std::cout << std::fixed << std::setprecision(2);

    // Z argumentami: jeden eksperyment (do strojenia i badań parametrów)
    if (argc > 1) {
        try {
            return runSingleExperiment(argc, argv);
        } catch (const std::exception& e) {
            std::cerr << "Blad: " << e.what() << '\n';
            return 1;
        }
    }

    // Bez argumentów: testy + pełne porównanie metod
    ...
}
```

`argc` to liczba argumentów (razem z nazwą programu), `argv` to ich teksty. `argc > 1` oznacza, że użytkownik podał jakieś argumenty.

Tryb 1: bez argumentów (pełne porównanie)

1. **`testTinyInstance()`**: liczy `G` na `tests/tiny.ttp` i porównuje z ręcznie policzonym `-49.25` (z tolerancją, bo liczby `double` nie porównuje się przez `==`).
2. **`testOperators()`**: przykłady swap/inwersji z treści, OX/PMX z oczekiwanym wynikiem i 10 000 losowych krzyżowań i mutacji sprawdzanych przez `isValidRoute`.
3. Jeśli któryś test nie przejdzie, program kończy działanie, zanim zacznie liczyć.
4. **Porównanie** na 7 instancjach, z **budżetem 10 000 ocen** na uruchomienie:

Metoda w CSV	Konfiguracja
`random`	10 000 losowych tras
`greedy`	najbliższy sąsiad z każdego z 280 miast
`ea_start`, `sa_start`	parametry startowe z treści (stan "przed strojeniem")
`ea`	`greedyInit=0.5`, swap `Pm=0.002`, `maxEvaluations=10000`
`sa`	swap, start zachłanny, `T0=10`, `iterations=9999`

EA i SA uruchamiane są 10 razy (seedy 42..51) przez `runEaSeries` / `runSaSeries`. Te funkcje zbierają najlepsze `G` z każdego uruchomienia, liczą statystyki, zapisują historię i dopisują wiersz do `results/wyniki.csv`.

Tryb 2: z argumentami (jeden eksperyment)

```
travelling_thief_metaheuristics.exe --method ea --instance extra/a280_n837_uncorr-
similar-weights_05.ttp --gen 500 --mut swap --pm 0.005 --runs 10 --out tuning/moje.csv
--label test
```

Najważniejsze fragmenty `runSingleExperiment`:

```
    // 1. Argumenty "--klucz wartosc" do słownika
    std::map<std::string, std::string> args;
    for (int i = 1; i < argc; i++) {
        std::string key = argv[i];
        ...
        if (key.rfind("--", 0) != 0) {
            throw std::runtime_error("Oczekiwano argumentu --klucz, a jest: " + key);
        }
        if (i + 1 >= argc) {
            throw std::runtime_error("Brak wartosci dla " + key);
        }
        args[key.substr(2)] = argv[i + 1];
        i++;  // przeskakujemy wartość
    }
```

1. Argumenty przychodzą parami `--klucz wartość`. Zapisujemy je w `std::map` (słownik), np. `args["gen"] = "500"`. `key.substr(2)` obcina `--`.
2. Nieznane klucze (np. literówka `--metod`) dają czytelny błąd zamiast cichego zignorowania.
3. `getInt` / `getDouble` / `getText` zwracają wartość argumentu albo wartość domyślną, jeśli argumentu nie podano. Dzięki temu każda konfiguracja startuje z wartości z `Config.h`, a zmieniamy tylko to, co podamy.
4. `parseSelection`, `parseCrossover`, `parseMutation` zamieniają tekst (`"pmx"`) na enum.
5. Wybrana metoda działa `--runs` razy, wynik idzie na ekran i jest **dopisywany** (`std::ios::app`) do pliku CSV. Nagłówek CSV zapisujemy tylko wtedy, gdy plik jest nowy.

`--help` wypisuje wszystkie opcje.

---

22. run_experiments.py: serie eksperymentów

```
EXPERIMENTS = {
    "ea_tour":   ("ea", "tour", [1, 2, 5, 10, 20, 50, 100], {}),
    "ea_budget": ("ea", "pop x gen", [{"pop": 20, "gen": 500}, ..., {"pop": 500, "gen":
20}], {}),
    ...
}
```

Każdy eksperyment to: metoda, zmieniany parametr, lista wartości i stałe dodatkowe parametry. Zmieniamy **jeden parametr naraz**. Wartość może być też słownikiem kilku parametrów (`ea_budget`: populacja i pokolenia przy stałym budżecie).

Dla każdej wartości skrypt uruchamia program C++ przez `subprocess.run` z odpowiednimi argumentami (10 uruchomień), zbiera wiersze w `results/tuning/<eksperyment>__<instancja>.csv` i rysuje wykres "wartość parametru → avg ± std" z linią najlepszego wyniku zachłannego.

```
python scripts/run_experiments.py                                  # lista eksperymentów
python scripts/run_experiments.py ea_tour --on easy1 hard1         # na wybranych
instancjach
python scripts/run_experiments.py all --on all                     # wszystko
```

`find_executable()` szuka programu najpierw w `cmake-build-release`, potem w `cmake-build-debug` (z ostrzeżeniem, że będzie wolno).

Pakowanie "smart" (eksperyment)

W `packItems` jest opcjonalna kontrola opłacalności (`--packing smart`):

```
        if (profitCheck) {
            double dist = remainingDistance[positionInRoute[problem.items[k].city]];
            double speedBefore = vmax - currentWeight * (vmax - vmin) / W;
            double speedAfter = vmax - (currentWeight + weight) * (vmax - vmin) / W;
            double extraTime = dist / speedAfter - dist / speedBefore;

            if (problem.items[k].profit <= extraTime) {
                continue;  // nie opłaca się
            }
        }
```

Przedmiot bierzemy tylko, gdy zysk przewyższa szacowany dodatkowy czas jazdy do końca trasy. **W badaniach ten wariant nigdzie nie pomógł**, a na łatwych instancjach pogorszył wynik o ok. 3%. Oszacowanie jest zbyt pesymistyczne: liczy spowolnienie przy całej wadze już wybranych przedmiotów, a one zwykle leżą dalej na trasie i na początku odcinka jeszcze ich nie ma. Domyślnie używane jest pakowanie `simple`.

---

23. Wyniki

Budżet 10 000 ocen, średnia najlepszego `G` z 10 uruchomień (pełna tabela: `docs/wyniki/tabela.md`, badania: `docs/BADANIA.md`):

instancja	Losowy	Zachłanny best	EA start	SA start	EA dostrojony	SA dostrojony
easy1 (n279 uncorr)	-10 849	35 071	11 275	24 765	**35 883**	34 354
easy2 (n279 bsc)	-7 290	30 882	12 399	23 719	**31 823**	30 916
medium1 (n1395 usw)	358 514	446 832	402 035	418 907	**457 212**	456 180
medium2 (n1395 bsc)	685 295	745 530	715 048	728 867	**753 236**	751 549
hard1 (n2790 uncorr)	1 283 352	1 363 918	1 325 077	1 344 272	**1 361 980**	1 360 805
hard2 (n2790 bsc)	2 844 927	2 921 166	2 882 361	2 899 193	**2 921 840**	2 919 716
extra (n837 usw)	191 332	267 553	226 396	243 523	**271 683**	269 882

Najważniejsze wnioski:

1. Przed strojeniem: zachłanny > SA > EA > losowy.
2. Po strojeniu EA i SA mają średnią wyższą niż średnia zachłannego na wszystkich instancjach, a średnia EA przebija nawet najlepszy wynik zachłannego na 6 z 7 instancji.
3. Największy efekt daje **start z tras zachłannych**, potem **mutacja swap z małym Pm** (0.001-0.005).
4. Turniej z `Tour ≥ 2` jest lepszy od ruletki, krzyżowanie jest kluczowe (`Px = 0` najgorsze).
5. Na trudnych instancjach różnice są względnie małe, bo zysk z przedmiotów (miliony) dominuje nad czasem podróży.

---

24. Założenia

- **Miasto startowe:** złodziej z niego wyrusza, a nie do niego "wchodzi", więc nie zbiera tam przedmiotów (dosłowne odczytanie treści).
- **`RENTING RATIO`:** wczytywany, ale pomijany w funkcji oceny, zgodnie z treścią.
- **Plan przedmiotów:** wyznacza go heurystyka `zysk / (waga × droga_do_końca)`. Osobnik w EA i rozwiązanie w SA to sama trasa.
- **Instancje:** 7 plików z rodziny `a280` (280 miast); trudność rośnie z liczbą przedmiotów (279 → 2790), typem plecaka i pojemnością.
- **Porównanie metod:** ten sam budżet 10 000 ocen funkcji celu dla losowego, EA i SA; 10 uruchomień EA i SA z seedami 42..51.

---

25. Pytania, które mogą paść na obronie

Dlaczego C++? Funkcja oceny jest wywoływana setki tysięcy razy. W C++ jedna ocena to ułamek milisekundy, w czystym Pythonie byłaby kilkadziesiąt razy wolniejsza. Python służy tylko do wykresów i tabel.

Jak reprezentujesz osobnika? Jako permutację miast (`std::vector<int>`). Plan przedmiotów nie jest częścią genotypu, tylko jest dobierany heurystyką przy każdej ocenie (transformacja TSP → TTP, jak sugeruje treść).

Dlaczego `std::vector`? Elementy leżą w pamięci obok siebie, dostęp po indeksie jest natychmiastowy, a kopiowanie to jedno szybkie skopiowanie bloku pamięci. Lista (`std::list`) byłaby wolniejsza.

Kopia głęboka czy płytka? Dziecko musi być kopią głęboką, bo potem je mutujemy, a rodzic nie może się zmienić. Selekcja nie kopiuje rodziców, tylko zwraca indeksy.

Jak liczysz odległości? Raz na starcie, `ceil` z odległości euklidesowej (`CEIL_2D`), w płaskiej tablicy `n×n`.

Skąd wiesz, że funkcja oceny liczy dobrze? Test na `tests/tiny.ttp` porównuje wynik z ręcznie policzonym `-49.25` przy każdym uruchomieniu programu.

Skąd wiesz, że krzyżowanie nie psuje tras? `isValidRoute` sprawdza 10 000 losowych dzieci przy każdym uruchomieniu.

Dlaczego porównujesz po liczbie ocen, a nie po czasie? Tak każe treść, a liczba ocen nie zależy od komputera. Czas też jest zapisywany w CSV.

Czemu wyniki są powtarzalne? Każde uruchomienie ma stały seed (`std::mt19937`), więc ten sam seed daje te same liczby na każdym komputerze z tym samym kompilatorem.

Dlaczego ruletka działa słabo? Ujemne `G` wymaga przesunięcia, a przy dużych wartościach `G` różnice między osobnikami są małe, więc prawdopodobieństwa wyboru są prawie równe i presja selekcyjna znika.